

Taller de Programación 2, I-2026

Patrones de Software y Programación

Sistema Web Autoadaptativo usando MAPE-K

Entrega: 09-06-2026 / Campus Virtual

1. Contexto

Una plataforma web ofrece contenido educativo a sus usuarios. En condiciones normales, el sitio muestra contenido enriquecido, incluyendo texto, imágenes, videos y otros recursos multimedia. Sin embargo, cuando aumenta considerablemente la demanda del sistema, por ejemplo, cuando muchos usuarios acceden al mismo tiempo, la plataforma debe adaptarse automáticamente para reducir el consumo de recursos.

En este taller, cada grupo deberá implementar un sistema web autoadaptativo basado en la arquitectura **MAPE-K**. El sistema deberá observar su propio estado, analizar el nivel de demanda, planificar una acción de adaptación y ejecutarla modificando el tipo de contenido que se muestra al usuario.

2. Descripción del sistema

Cada grupo deberá implementar en **Java** un sistema web autoadaptativo utilizando **Maven** como gestor de dependencias y construcción, y **Javalin** como tecnología web.

El sistema debe mostrar contenido educativo mediante un endpoint principal. Según el nivel de demanda detectado, deberá cambiar automáticamente entre tres modos de presentación:

- **Modo multimedia:** muestra título, descripción, imagen, video o enlace multimedia.
- **Modo restringido:** muestra título, descripción e imágenes, pero desactiva videos o enlaces multimedia.
- **Modo texto:** muestra título, descripción resumida y un mensaje indicando que las imágenes, videos y enlaces multimedia fueron desactivados temporalmente debido a alta demanda.

El componente **Knowledge** deberá almacenar el estado actual del sistema y los parámetros de configuración. Este componente podrá implementarse mediante un archivo de configuración, un archivo JSON, un archivo `.properties` o una base de datos.

El uso de **Javalin** permitirá implementar un servidor web liviano en Java, definiendo al menos los siguientes endpoints:

```
http://localhost:7000/content
http://localhost:7000/status
http://localhost:7000/reset
```

El uso de **Maven** permitirá administrar las dependencias del proyecto y ejecutar la aplicación de forma estandarizada. Cada grupo deberá entregar un proyecto Java Maven funcional, con su respectivo archivo `pom.xml`.

3. Arquitectura MAPE-K

El sistema debe implementar explícitamente los componentes de la arquitectura MAPE-K:

- **Monitor:** registra información sobre el estado del sistema, por ejemplo, la cantidad de solicitudes recibidas.
- **Analyze:** determina el nivel de demanda a partir de los datos recopilados por el Monitor y los umbrales definidos en Knowledge.
- **Plan:** decide la acción de adaptación que debe aplicarse según el nivel de demanda detectado.
- **Execute:** ejecuta la adaptación, cambiando el modo de funcionamiento del sistema.
- **Knowledge:** almacena el modo actual, los umbrales de carga y otros parámetros necesarios para la adaptación.

Se sugiere organizar el proyecto de la siguiente manera:

```
src/main/java/  
|-- app/           Main.java  
|-- web/           WebServer.java, ContentController.java  
|-- mape/          Monitor.java, Analyzer.java, Planner.java, Executor.java  
|-- knowledge/     KnowledgeBase.java, SystemState.java, AdaptationConfig.java  
|-- model/         Content.java  
|-- service/       ContentService.java
```

Esta estructura puede ser modificada por cada grupo, siempre que se mantenga una separación clara entre los componentes MAPE-K y el resto del sistema.

4. Funcionamiento esperado

El sistema debe ejecutar el ciclo MAPE-K durante su funcionamiento. Cada vez que se reciba una solicitud al endpoint principal, el sistema deberá registrar dicha solicitud, analizar el nivel de demanda y decidir si corresponde mantener el modo actual o activar otro modo de presentación.

El flujo esperado es el siguiente:

1. Un usuario realiza una solicitud al endpoint `/content`.
2. El componente **Monitor** registra la solicitud.
3. El componente **Analyze** compara la cantidad de solicitudes con los umbrales definidos.
4. El componente **Plan** decide si corresponde activar el modo multimedia, restringido o texto.
5. El componente **Execute** aplica la adaptación correspondiente.
6. El componente **Knowledge** actualiza o mantiene el estado actual del sistema.
7. El contenido se muestra según el modo activo.

En consola o en un archivo de registro, el sistema deberá mostrar evidencia del proceso de adaptación. Por ejemplo:

```
[MONITOR] Solicitud registrada. Total actual: 10
[ANALYZE] Demanda media detectada.
[PLAN] Activar modo restringido.
[EXECUTE] El sistema cambio a modo RESTRICTED.
```

```
[MONITOR] Solicitud registrada. Total actual: 20
[ANALYZE] Alta demanda detectada.
[PLAN] Activar modo texto.
[EXECUTE] El sistema cambio a modo TEXT.
```

5. Simulación de carga mediante herramienta externa

Cada grupo deberá demostrar la autoadaptación del sistema utilizando una herramienta externa para generar múltiples peticiones HTTP hacia el servidor web. La prueba de carga debe realizarse sobre el endpoint principal:

```
http://localhost:7000/content
```

La herramienta externa puede ser una de las siguientes: **Apache Bench**, **curl**, **Postman Collection Runner**, **JMeter** o **k6**.

Para este taller, el sistema deberá considerar al menos dos umbrales de carga definidos en el componente **Knowledge**. Estos umbrales permitirán activar tres modos de funcionamiento. Por ejemplo:

```
restricted.threshold = 10
text.threshold = 20
```

Con estos valores, el comportamiento esperado será:

- entre 0 y 9 solicitudes, el sistema funciona en **modo multimedia**;
- entre 10 y 19 solicitudes, el sistema cambia a **modo restringido**;
- desde 20 solicitudes en adelante, el sistema cambia a **modo texto**.

La prueba externa deberá enviar una cantidad de solicitudes suficiente para evidenciar al menos un cambio de modo. Idealmente, deberá mostrar el paso desde el modo multimedia al modo restringido y luego al modo texto.

El endpoint **/status** deberá permitir consultar el estado actual del sistema, incluyendo al menos el modo activo, la cantidad de solicitudes registradas y los umbrales configurados. Una posible respuesta es:

```
{
  "mode": "TEXT",
  "requests": 25,
  "restrictedThreshold": 10,
  "textThreshold": 20
}
```

El endpoint **/reset** deberá permitir reiniciar la simulación, dejando el contador de solicitudes en cero y restaurando el sistema al modo multimedia.

6. Entregables

Cada grupo deberá entregar:

1. Código fuente completo del sistema.
2. Archivo `pom.xml` con las dependencias necesarias para ejecutar Javalin.
3. Archivo `README.md` con:
 - descripción general del sistema;
 - instrucciones de instalación y ejecución;
 - explicación de los componentes MAPE-K implementados;
 - descripción del mecanismo usado para Knowledge;
 - herramienta externa utilizada para generar carga;
 - comando o configuración utilizada para la prueba de carga;
 - evidencia de que el sistema cambia entre los tres modos de presentación.
4. Diagrama de arquitectura del sistema.
5. Capturas de pantalla del sistema en modo multimedia, restringido y texto.
6. Registro de consola o archivo log donde se evidencie el ciclo MAPE-K.

7. Criterios de evaluación

Criterio	Puntaje
Implementa correctamente el servidor web usando Java, Maven y Javalin	10 pts
Implementa claramente los componentes Monitor, Analyze, Plan y Execute	30 pts
Implementa el componente Knowledge mediante archivo o base de datos	15 pts
El sistema cambia automáticamente entre modo multimedia, restringido y texto	15 pts
Demuestra la autoadaptación mediante una herramienta externa de carga	10 pts
Presenta evidencia de pruebas, capturas y registros del ciclo MAPE-K	10 pts
Código organizado, modular y comprensible	5 pts
README claro y completo	5 pts
Total	100 pts